What is a NULL Value?

A **NULL value** represents the absence of a known or meaningful value for a particular field.

For example, suppose a college maintains student records.

Roll No.	Name	Course	Email
101	Rahul	BCA	rahul@gmail.com
102	Priya	BCA	null
103	Amit	BCA	amit@gmail.com
104	Neha	BCA	null

In this example, Rahul and Amit have email addresses, but Priya and Neha have `null` as their email value.

In MongoDB, we can store `null` explicitly:

```
{
    rollno: 102,
    name: "Priya",
    course: "BCA",
    email: null
}
```

This means the `email` field exists, but its value is explicitly set to `null`.

Important: `null` is not the same as `0`, an empty string (`""`), or a missing field.

Difference Between NULL, Empty String and Missing Field

Consider these four MongoDB documents:

Rahul - `"rahul@gmail.com"` Email address is available

```
{ name: "Rahul", email: "rahul@gmail.com" }
```

Rahul - `"rahul@gmail.com"` Email address is available

```
{ name: "Priya", email: null }
```

Amit - `""` Field exists with an empty string

```
{ name: "Amit", email: "" }
```

Neha - Field missing Email field has not been stored

```
{ name: "Neha" }
```

Practical 1: Find Documents Where a Field Is NULL

```
db.nullStudents.find({
         email: null
})
```

Practical 2: Find Only Documents Where the Field Exists and Is NULL

```
db.nullStudents.find({
        email: {
                $eq: null,
                $exists: true
                }
})
```

Practical 3: Find Documents Where a Field Is Missing

```
db.nullStudents.find({
        email: {
                $exists: false
        }
})
```

Practical 4: Find Documents Where a Field Is NOT NULL

```
db.nullStudents.find({
        email: {
                $ne: null
        }
})
```

Practical 5: Find Empty String Values

```
db.nullStudents.find({
          email: ""
})
```

Practical 6: Update NULL Values Using $set

```
db.nullStudents.updateOne(
        {
                rollno: 102
        },
        {
                $set: {
                        email: "priya@gmail.com"
                }
        }
)
```

Practical 7: Remove a Field Using $unset

```
db.nullStudents.updateOne(
        {
                rollno: 104
        },
        {
                $unset: {
                        email: ""
                }
        }
)
```

```
Practical 8: Update All Documents Where a Field Is NULL

        db.nullStudents.updateMany(
                {
                        email: {
                                $eq: null,
                                $exists: true
                        }
                },
                {
                        $set: {
                                emailStatus: "Not Provided"
                        }
                }
        )
```

What is Count?

The **Count operation** is used to determine how many documents satisfy a particular condition in a collection.

For example:

- How many students are registered?
- How many students are studying in Semester 5?
- How many students scored more than 80 marks?
- How many students belong to Anjar?

MongoDB provides the `countDocuments()` method for counting documents.

Syntax

```
db.collectionName.countDocuments()
```

To count documents matching a condition:

```
db.collectionName.countDocuments({fieldName: value})
```

Practical Examples

Example 1: Count all students

```
db.students.countDocuments({})
```

Expected output:

```
8
```

Explanation: The empty filter `{}` matches every document in the collection.

Example 2: Count students studying in Semester 5

```
db.students.countDocuments({semester: 5})
```

Expected output:

```
5
```

Example 3: Count students from Anjar

```
db.students.countDocuments({city: "Anjar"})
```

Expected output:

```
3
```

Example 4: Count students who scored more than 80 marks

```
db.students.countDocuments({marks: {$gt: 80}})
```

Expected output:

```
4
```

Here, `$gt` means **greater than**.

Example 5: Count students who scored less than 70 marks

```
db.students.countDocuments({marks: {$lt: 70}})
```

Expected output:

```
2
```

Here, `$lt` means **less than**.

Example 6: Count students whose marks are between 70 and 90, inclusive

```
db.students.countDocuments({
  marks: {$gte: 70, $lte: 90}
})
```

Expected output:

```
4
```

Here:

- `$gte` means greater than or equal to.

- `$lte` means less than or equal to.

Example 7: Count students studying in Semester 5 who scored above 80

```
db.students.countDocuments({
  semester: 5,
  marks: {$gt: 80}
})
```

Expected output:

```
4
```

Explanation: Both conditions must be satisfied.

Example 8: Count students from Bhuj

```
db.students.countDocuments({city: "Bhuj"})
```

Expected output:

```
3
```

2.4 Important Operators Used with Count

Operator	Meaning	Example
`$gt`	Greater than	`{marks: {$gt: 80}}`
`$gte`	Greater than or equal to	`{marks: {$gte: 80}}`
`$lt`	Less than	`{marks: {$lt: 70}}`
`$lte`	Less than or equal to	`{marks: {$lte: 70}}`
`$ne`	Not equal to	`{city: {$ne: "Anjar"}}`
`$in`	Matches any value in a list	`{city: {$in: ["Anjar", "Bhuj"]}}`

Example 9: Count students who are not from Anjar

```
db.students.countDocuments({
  city: {$ne: "Anjar"}
})
```

Expected output:

```
5
```

Example 10: Count students from either Anjar or Bhuj

```
db.students.countDocuments({
  city: {$in: ["Anjar", "Bhuj"]}
})
```

Expected output:

What is Limit?

The **limit() method** restricts the number of documents returned by a query.

Suppose a college has 5,000 student records, but you want to display only the first 5 students. You can use limit(5).

Syntax

db.collectionName.find().limit(number)

Practical Examples

Example 1: Display only 3 students

```
db.students.find().limit(3)
```

This returns at most three documents.

Example 2: Display only 5 students

```
db.students.find().limit(5)
```

This returns at most five documents.

Example 3: Display only 3 students from Bhuj

```
db.students.find({city: "Bhuj"}).limit(3)
```

This returns at most three matching students.

Example 4: Display only 2 students who scored more than 80

```
db.students.find({
  marks: {$gt: 80}
}).limit(2)
```

This returns at most two matching documents.

Important: Without sorting, you should not depend on which matching documents appear first.

Example 5: Display the 3 highest-scoring students

```
db.students.find()
  .sort({marks: -1})
```

```
  .limit(3)
```

Expected student names and marks:

Name	Marks
Pooja	95
Priya	92
Meera	88

Explanation:

- `sort({marks: -1})` sorts marks from highest to lowest.
- `limit(3)` returns only the first three documents after sorting.

Example 6: Display the 2 lowest-scoring students

```
db.students.find()
  .sort({marks: 1})
  .limit(2)
```

Expected student names and marks:

Name	Marks
Karan	55
Amit	67

Why is Limit Useful?

Limit is commonly used for:

- Displaying a fixed number of records.
- Showing top-performing students.
- Displaying a preview of data.
- Reducing the number of documents returned to an application.

What is Sort?

The **`sort()` method** arranges documents according to one or more fields.

For example:

- Arrange students alphabetically.
- Sort students according to marks.
- Display the oldest students first.
- Arrange students by semester and marks.

Syntax

```
db.collectionName.find().sort({fieldName: 1})
```

For descending order:

```
db.collectionName.find().sort({fieldName: -1})
```

Remember

Value	Meaning
`1`	Ascending order
`-1`	Descending order

For numeric fields:

- Ascending: lowest to highest.
- Descending: highest to lowest.

For strings:

- Ascending generally means alphabetical order.
- Descending generally means reverse alphabetical order.

Practical Examples

Example 1: Sort students by marks in ascending order

```
db.students.find().sort({marks: 1})
```

Expected order:

```
Karan  - 55
Amit   - 67
Jay    - 73
Neha   - 78
Rahul  - 85
Meera  - 88
Priya  - 92
Pooja  - 95
```

Example 2: Sort students by marks in descending order

```
db.students.find().sort({marks: -1})
```

Expected order:

```
Pooja  - 95
Priya  - 92
Meera  - 88
Rahul  - 85
Neha   - 78
Jay    - 73
Amit   - 67
```

```
Karan  - 55
```

Example 3: Sort students alphabetically by name

```
db.students.find().sort({name: 1})
```

This arranges names in ascending alphabetical order.

Example 4: Sort students by age in descending order

```
db.students.find().sort({age: -1})
```

This displays older students first.

Example 5: Sort students by semester

```
db.students.find().sort({semester: 1})
```

Students in Semester 3 appear before students in Semester 5.

Example 6: Sort by semester and then by marks

```
db.students.find().sort({
  semester: 1,
  marks: -1
})
```

Explanation:

1. Students are grouped in ascending semester order.
2. Within each semester, students are sorted from highest marks to lowest.

Example 7: Sort by city and name

```
db.students.find().sort({
  city: 1,
  name: 1
})
```

This sorts by city first and then by name within each city.

Example 8: Display the highest-scoring student

```
db.students.find()
  .sort({marks: -1})
  .limit(1)
```

Expected result:

```
Pooja - 95 marks
```

What is Skip?

The **`skip()` method** skips a specified number of documents before returning the remaining matching documents.

For example, if a query returns eight documents and you use `skip(3)`, MongoDB skips the first three documents and returns the remaining five, subject to any other query options.

Skip is commonly used for pagination.

Pagination means dividing a large result set into smaller pages.

For example:

- Page 1: Students 1–3
- Page 2: Students 4–6
- Page 3: Students 7–9

Syntax

```
db.collectionName.find().skip(number)
```

Practical Examples

Example 1: Skip the first 3 students

```
db.students.find().skip(3)
```

This skips three documents and returns the remaining documents.

Example 2: Skip the first 5 students

```
db.students.find().skip(5)
```

This skips five documents.

Example 3: Skip 2 students and return the next 3

```
db.students.find().skip(2).limit(3)
```

This skips two documents and returns at most three documents after them.

Example 4: Display students after the first 3 in marks order

```
db.students.find()
  .sort({marks: -1})
  .skip(3)
```

This sorts students by marks from highest to lowest, skips the first three, and returns the rest.

Example 5: Display the second page with 3 students per page

```
db.students.find()
  .sort({rollno: 1})
  .skip(3)
  .limit(3)
```

Expected student roll numbers:

```
104
105
106
```

Explanation:

The first page contains roll numbers 101, 102, and 103. The second page skips those three documents and returns the next three.

What is an Array?

An **array** is a collection of multiple values stored in a single field of a MongoDB document.

For example, one student can study multiple subjects. Instead of creating separate subject fields, we can store the subjects in an array.

```
{
  rollno: 101,
  name: "Rahul",
  subjects: ["Java", "MongoDB", "Python"]
}
```

Here, `subjects` is an array containing three values.

MongoDB supports arrays containing strings, numbers, objects, and other data types.

6.2 Why Are Arrays Used?

Arrays are useful when one document has multiple related values, such as:

- Subjects studied by a student.
- Skills possessed by an employee.
- Items in a shopping cart.
- Phone numbers associated with a contact.
- Tags assigned to a blog post.

6.3 View Student Arrays

Display only all students name and their subjects:

```
db.students.find({}, {
  name: 1,
  subjects: 1
})
```

Explanation:

- `name: 1` includes the name.
- `subjects: 1` includes the subjects array.

6.4 Find Documents Containing a Particular Array Element

Example 1: Find students studying MongoDB

```
db.students.find({
  subjects: "MongoDB"
})
```

MongoDB matches documents where the `subjects` array contains `"MongoDB"`.

ARRAY OPERATORS in MongoDB

MongoDB provides several operators to query and modify arrays.

The important operators for practical examinations are:

Operator	Purpose
`$all`	Match arrays containing all specified elements
`$in`	Match a field against any value in a list
`$size`	Match arrays having an exact number of elements
`$push`	Add an element to an array
`$addToSet`	Add an element only if it is not already present
`$pull`	Remove matching elements from an array
`$pop`	Remove the first or last array element
`$elemMatch`	Match array elements satisfying specified conditions

$all Operator

The $all operator finds documents whose array contains **all the specified values**.

Example: Find students studying both Python and Java

db.students.find({

```
  subjects: {$all: ["Python", "Java"]}
})
```

$in Operator

The $in operator matches documents where the field value matches any value in a specified list.

For array fields, it can match documents where the array contains any of the listed elements.

Example: Find students studying either Java or PHP

```
db.students.find({
  subjects: {$in: ["Java", "PHP"]}
})
```

$size Operator

The $size operator matches arrays containing exactly the specified number of elements.

Example: Find students who have exactly 2 subjects

```
db.students.find({
  subjects: {$size: 2}
})
```

$push Operator

The $push operator adds a new value to an array.

Example: Add PHP to Rahul's subjects

```
db.students.updateOne(
  {rollno: 101},
  {$push: {subjects: "PHP"}}
)
```

Check the result:

```
db.students.findOne({rollno: 101})
```

$addToSet Operator

The $addToSet operator adds a value only when that value is not already present in the array.

Example: Add MongoDB to Rahul's subjects

```
db.students.updateOne(
  {rollno: 101},
  {$addToSet: {subjects: "MongoDB"}}
)
```

Rahul already studies MongoDB, so this operation will not add a duplicate.

Example: Add Data Science to Rahul's subjects

```
db.students.updateOne(
  {rollno: 101},
  {$addToSet: {subjects: "Data Science"}}
)
```

The new subject is added because it is not already present.

$pull Operator

The $pull operator removes all array elements that match a specified value or condition.

Example: Remove Python from Rahul's subjects

```
db.students.updateOne(
  {rollno: 101},
  {$pull: {subjects: "Python"}}
)
```

Check:

```
db.students.findOne({rollno: 101})
```

Python will be removed from Rahul's subjects.

$pop Operator

The $pop operator removes an element from one end of an array.

- $pop: {field: 1} removes the last element.
- $pop: {field: -1} removes the first element.

Example: Remove the last subject from Rahul's array

```
db.students.updateOne(
  {rollno: 101},
  {$pop: {subjects: 1}}
)
```

Example: Remove the first subject from Amit's array

```
db.students.updateOne(
  {rollno: 103},
  {$pop: {subjects: -1}}
)
```

$elemMatch Operator

The $elemMatch operator is useful when an array contains objects and you want to match an element satisfying multiple conditions.

Let's create an additional collection for this exercise.

```
db.examResults.insertMany([
  {
    rollno: 101,
```

```
    name: "Rahul",
    results: [
      {subject: "Java", marks: 85},
      {subject: "MongoDB", marks: 90}
    ]
  },
  {
    rollno: 102,
    name: "Priya",
    results: [
      {subject: "Java", marks: 60},
      {subject: "MongoDB", marks: 95}
    ]
  },
  {
    rollno: 103,
    name: "Amit",
    results: [
      {subject: "Java", marks: 75},
      {subject: "MongoDB", marks: 70}
    ]
  }
])
```

Example: Find students who scored at least 80 in Java

```
db.examResults.find({
```

```
  results: {
    $elemMatch: {
      subject: "Java",
      marks: {$gte: 80}
    }
  }
})
```

What is Aggregation?

Aggregation is a MongoDB feature used to process documents, perform calculations, group records, and produce summarized results.

Aggregation is useful for answering questions such as:

- What is the average student score?
- What is the highest mark?
- How many students are in each semester?
- What are the total and average marks for each semester?
- How many students belong to each city?
- Which semester has the highest average score?

MongoDB aggregation is commonly performed using an **aggregation pipeline**.

8.2 What is an Aggregation Pipeline?

An aggregation pipeline consists of multiple stages. Each stage processes documents and passes its output to the next stage.

Common stages include:

Stage	Purpose
`$match`	Filters documents.
`$group`	Groups documents and calculates summary values.
`$sort`	Sorts documents.
`$limit`	Restricts the number of results.
`$skip`	Skips documents.
`$project`	Selects, removes, or computes output fields.
`$unwind`	Creates a separate pipeline document for each array element.
`$count`	Counts documents reaching that stage.

AGGREGATION OPERATORS

Before performing aggregation exercises, learn the following operators.

Operator	Meaning
`$sum`	Calculates a total or count when used appropriately.
`$avg`	Calculates an average.
`$min`	Finds the minimum value.
`$max`	Finds the maximum value.
`$first`	Returns the first value in a group.
`$last`	Returns the last value in a group.
`$push`	Collects values into an array.
`$addToSet`	Collects unique values into an array.

AGGREGATION PRACTICALS

Practical 1: Count All Students Using Aggregation

```
db.students.aggregate([
  {$count: "totalStudents"}
])
```

Expected output:

```
[
  {
    totalStudents: 8
  }
]
```

Explanation: `$count` counts the documents that reach this stage and stores the result under `totalStudents`.

If no documents reach the stage, the pipeline returns no count document.

Practical 2: Calculate the Total Marks

```
db.students.aggregate([
  {
    $group: {
      _id: null,
      totalMarks: {$sum: "$marks"}
    }
  }
])
```

Expected output:

```
[
  {
    _id: null,
    totalMarks: 633
  }
]
```

Explanation:

- `_id: null` places all students into one group.
- `$sum: "$marks"` adds the marks field from the documents in that group.
- `$` before `marks` indicates that the value comes from the document's `marks` field.

Practical 3: Calculate the Average Marks

```
db.students.aggregate([
  {
    $group: {
      _id: null,
      averageMarks: {$avg: "$marks"}
    }
  }
])
```

Expected output:

```
[
  {
    _id: null,
    averageMarks: 79.125
  }
]
```

This calculates the average marks of all eight students.

Practical 4: Find the Highest Marks

```
db.students.aggregate([
  {
    $group: {
      _id: null,
      highestMarks: {$max: "$marks"}
    }
  }
])
```

Expected result:

```
{
  _id: null,
  highestMarks: 95
}
```

Practical 5: Find the Lowest Marks

```
db.students.aggregate([
  {
    $group: {
      _id: null,
      lowestMarks: {$min: "$marks"}
    }
  }
])
```

Expected result:

```
{
  _id: null,
  lowestMarks: 55
}
```

Practical 6: Calculate Total Students in Each Semester

```
db.students.aggregate([
  {
    $group: {
      _id: "$semester",
      totalStudents: {$sum: 1}
    }
  },
  {
    $sort: {_id: 1}
  }
])
```

Expected output:

Semester	Total Students
3	3
5	5

Explanation:

- `_id: "$semester"` groups students according to semester.
- `$sum: 1` adds one for each document in the group.
- `$sort` arranges the results by semester.

Practical 7: Calculate Average Marks for Each Semester

```
db.students.aggregate([
  {
    $group: {
```

```
      _id: "$semester",
      averageMarks: {$avg: "$marks"}
    }
  },
  {
    $sort: {_id: 1}
  }
])
```

Expected output, rounded for display:

Semester	Average Marks
3	72.67
5	83.00

The actual result may display more decimal places.

Practical 8: Calculate Total Marks for Each Semester

```
db.students.aggregate([
  {
    $group: {
      _id: "$semester",
      totalMarks: {$sum: "$marks"}
    }
  },
  {
    $sort: {_id: 1}
  }
])
```

Expected output:

Semester	Total Marks
3	218
5	415

Practical 9: Count Students in Each City

```
db.students.aggregate([
  {
    $group: {
      _id: "$city",
      totalStudents: {$sum: 1}
    }
  },
  {
    $sort: {totalStudents: -1, _id: 1}
  }
])
```

Expected output:

City	Total Students

Anjar	3
Bhuj	3
Gandhidham	2

The city names are used as the group identifiers. The secondary sort by city makes the output predictable when counts are tied.

Practical 10: Find Highest and Lowest Marks Together

```
db.students.aggregate([
  {
    $group: {
      _id: null,
      highestMarks: {$max: "$marks"},
      lowestMarks: {$min: "$marks"},
      averageMarks: {$avg: "$marks"},
      totalMarks: {$sum: "$marks"}
    }
  }
])
```

This single query calculates:

- Highest marks.
- Lowest marks.
- Average marks.
- Total marks.

Practical 11: Find the Top 3 Students Using Aggregation

```
db.students.aggregate([
  {$sort: {marks: -1, rollno: 1}},
  {$limit: 3},
  {
    $project: {
      _id: 0,
      rollno: 1,
      name: 1,
      marks: 1
    }
  }
])
```

Expected results:

Roll No.	Name	Marks
108	Pooja	95
102	Priya	92
106	Meera	88

Explanation:

1. `$sort` arranges students by marks in descending order.

2. `rollno: 1` provides a tie-breaker if students have equal marks.
3. `$limit` keeps the first three documents.
4. `$project` displays only the required fields and excludes `_id`.

Practical 12: Find Semester 5 Students with Marks Above 80

```
db.students.aggregate([
  {
    $match: {
      semester: 5,
      marks: {$gt: 80}
    }
  },
  {
    $sort: {marks: -1}
  },
  {
    $project: {
      _id: 0,
      name: 1,
      semester: 1,
      marks: 1
    }
  }
])
```

This pipeline:

1. Selects Semester 5 students who scored above 80.
2. Sorts them from highest to lowest.
3. Displays only the required fields.

Practical 13: Display All Subjects Individually Using `$unwind`

The `$unwind` stage creates a separate pipeline document for each array element.

For example, Rahul's document contains:

```
subjects: ["Java", "MongoDB", "Python"]
```

After `$unwind`, the subjects are represented separately in the pipeline.

Command

```
db.students.aggregate([
  {$unwind: "$subjects"},
  {
    $project: {
      _id: 0,
      name: 1,
      subject: "$subjects"
    }
```

```
  }
])
```

Example output:

```
{ name: "Rahul", subject: "Java" }
{ name: "Rahul", subject: "MongoDB" }
{ name: "Rahul", subject: "Python" }
```

The remaining students' subjects will also be displayed as separate pipeline documents.

When is `$unwind` useful?

- Counting how many students study each subject.
- Analysing product categories in order records.
- Processing tags or skills stored in arrays.
- Reporting individual elements of an array.

Practical 14: Count Students Studying Each Subject

```
db.students.aggregate([
  {$unwind: "$subjects"},
  {
    $group: {
      _id: "$subjects",
      totalStudents: {$sum: 1}
    }
  },
  {
    $sort: {totalStudents: -1, _id: 1}
  }
])
```

Explanation:

1. `$unwind` separates each subject into its own pipeline document.
2. `$group` groups documents by subject.
3. `$sum: 1` counts each subject occurrence.
4. `$sort` arranges subjects by count, with alphabetical ordering as a tie-breaker.

This query counts subject occurrences across student documents. If a single student's array contains the same subject more than once, that student contributes more than one occurrence.

Practical 15: Group Students and Collect Their Names

```
db.students.aggregate([
  {
    $group: {
      _id: "$semester",
      studentNames: {$push: "$name"}
    }
  },
  {
```

```
    $sort: {_id: 1}
  }
])
```

This creates one result per semester, with an array of student names.

Example structure:

```
{
  _id: 3,
  studentNames: ["Amit", "Neha", "Jay"]
}
```

The actual order of names inside the array depends on the order of documents reaching `$group`. If a particular order is required, sort the input documents before grouping.

Difference Between Find and Aggregate

Feature	**find()**	**aggregate()**
Main purpose	Retrieve matching documents.	Process documents and produce results.
Filtering	Yes	Yes, using $match.
Sorting	Yes	Yes, using $sort.
Limiting results	Yes	Yes, using $limit.
Counting	Use countDocuments() for document counts.	Use $count or $group.
Average calculation	Not directly as a summary query.	Yes, using $avg.
Grouping	Not directly.	Yes, using $group.
Array expansion	No direct equivalent to $unwind.	Yes, using $unwind.